

GraphService: **Topology-aware Constructor** for Large-scale Graph Applications

A Topology-aware Constructor for Large-scale Graph Applications

Xinbiao Gan

National University of Defense Technology



汇报人：薛远识

CONTENTS

01  Research Background and Problem Statement

02  Related Technical Background

03  Core Design of GraphService

04  Experimental Evaluation and Results

05  Conclusion and Future Work

Research Background: The Importance of Graph Services



Widespread Application Scenarios

Graph services have been deeply integrated into daily life, covering social networks, recommendation systems, knowledge graphs, and other fields.



Key to Quality of Service (QoS)

Quality of Service (QoS) is a key metric for measuring graph services, including throughput, response time, etc.



Distributed Needs Driven by Massive Data

As the scale of graph data (vertices/edges) grows rapidly, a single machine can no longer handle it, and distributed systems are a must.

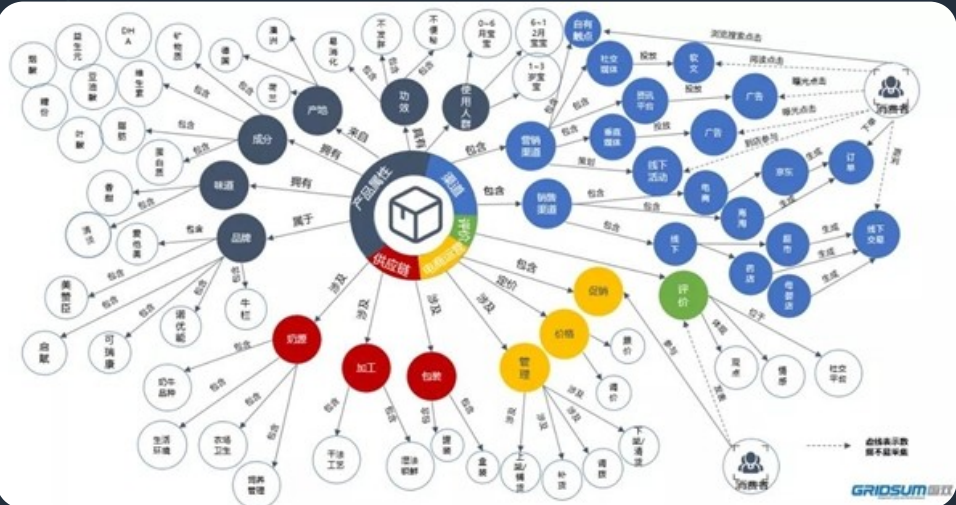


Figure: Schematic of a complex knowledge graph (showing scale and association)

Problem Statement: Bottleneck of Graph Construction



Traditional systems focus too much on **high throughput** of operators (BFS/SSSP), but ignore the huge overhead of the construction phase.

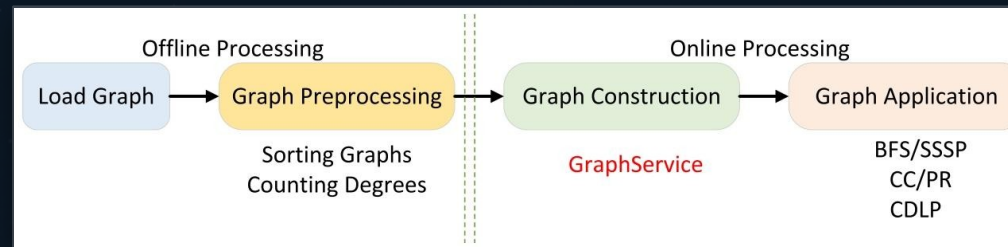


Graph construction time (τ_c) is typically **3-5 orders of magnitude** higher than operator time (τ_g), a key bottleneck.

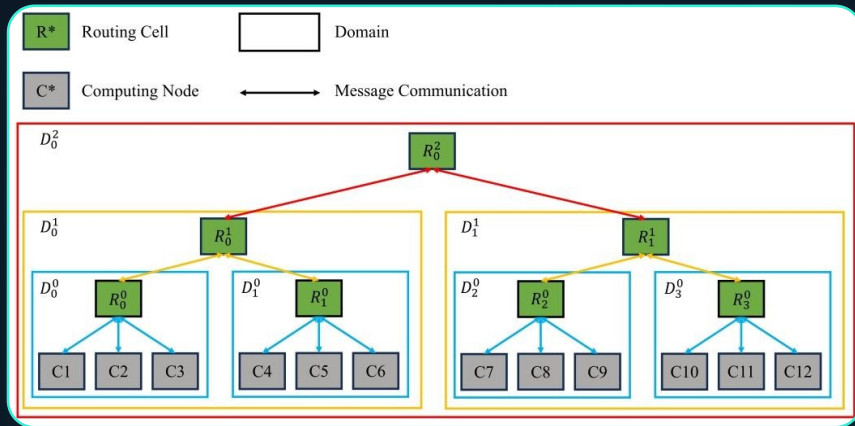


Communication costs account for **over 80%** of the construction phase, the core cause of high latency.

Graph Processing Pipeline: Construction is the critical entry point of online processing



Core Problem: Neglect of Communication Topology



Schematic of HPC Hierarchical Communication Topology



Algorithm Limitation: Ignoring Underlying Characteristics

Current graph partitioning algorithms focus only on the graph's sparsity, ignoring the physical communication hierarchy of the underlying HPC system.



Hardware Feature: Hierarchical Communication Latency

HPC networks have strict hierarchies; cross-cabinet/room latency is much higher than intra-level communication.



Performance Bottleneck: High Cross-domain Overhead

Neglecting topology causes unnecessary cross-domain traffic, drastically increasing latency and reducing efficiency.

Solution: Introduction to GraphService

To solve the above problems, we propose GraphService, a flexible and efficient graph constructor. Its core ideas include two aspects:

Hierarchy-aware Graph Partitioner

Utilizes the communication topology information of the target HPC system to distribute graph data as much as possible within the same communication domain, thereby minimizing communication costs.

Space-time Efficient Graph Compression

Accelerates graph construction and reduces memory usage by identifying and grouping a large number of vertices with the same degree in the graph for batch memory access.



GraphService Overall Architecture Diagram

Background: HPC Communication Topology

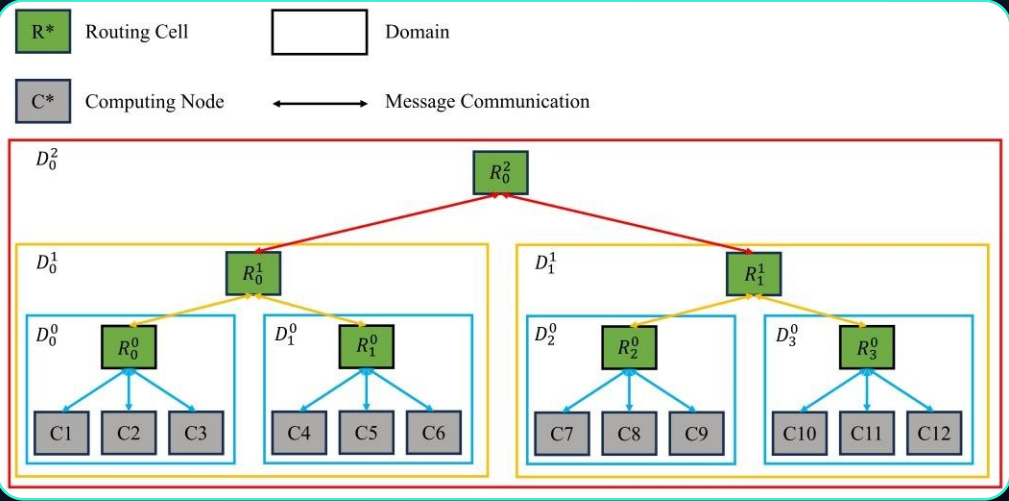


Figure 1: Hierarchical Communication Topology of HPC System



Hierarchical Connection Architecture

Large-scale HPC systems connect compute nodes through routing units (RC) to form hierarchical communication domains.



Multi-level Domain Division

Communication domains are divided into different levels, such as leaf domains, level-1 domains, level-2 domains, etc.



Communication Latency

Communication latency between nodes within the same communication domain is much lower than cross-domain communication.



Core Optimization Goal

The optimization goal is to place related graph data within the same communication domain to minimize communication overhead.

Background: CSR and Graph Distribution

$\begin{bmatrix} 1 & 7 & 0 & 0 \\ 0 & 2 & 8 & 0 \\ 5 & 0 & 3 & 9 \\ 0 & 6 & 0 & 4 \end{bmatrix}$	0 2 4 7 9	行偏移量(row offsets)
	0 1 1 2 0 2 3 1 3	非零元素列下标 (column indices)
	1 7 2 8 5 3 9 6 4	数值(values)

CSR (Compressed Sparse Row) storage format for sparse matrices



CSR Storage Standard

CSR is a standard format for storing sparse graph data, representing the graph through three arrays (val, col_idx, row_ptr).



Distributed Graph Partitioning

In distributed graph processing, vertices and edges are distributed to different compute nodes.



Key Performance Factors

Partitioning strategy determines communication patterns and efficiency, a critical step in distributed processing.

Core Design 1: Hierarchy-aware Graph Partitioner



Design Philosophy

Allocate vertices and edges of the graph as much as possible within the same communication domain to reduce cross-domain communication.



Specific Strategy

Uses HPC system's communication topology to guide partitioning, aligning results with hardware hierarchy.



Core Advantage

Reduces communication cost and time by minimizing expensive cross-domain communication.

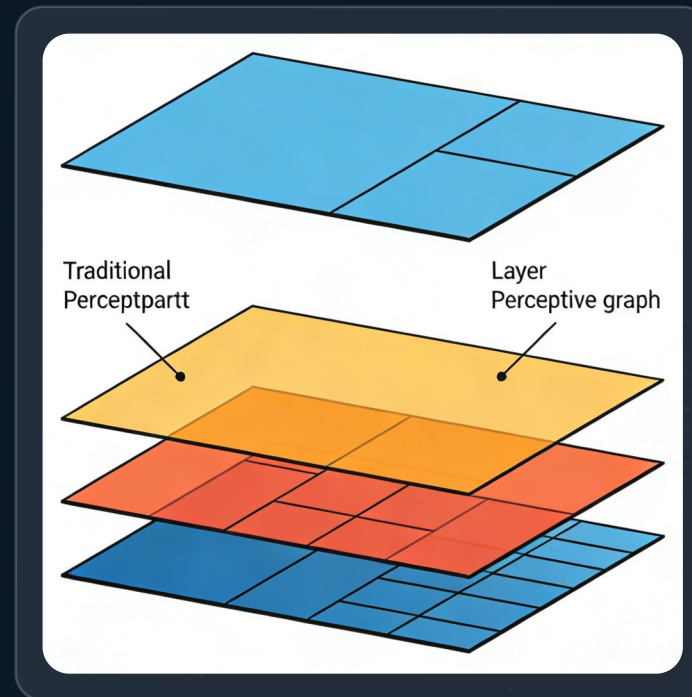


Fig: Hierarchy-aware vs Traditional Strategy

Core Design 2: Space-time Efficient Graph Compression



Key Observation

In real-world graphs, there are a large number of vertices with the same degree.



Compression Method

Groups vertices with the same degree for storage/indexing, avoiding duplicate structural metadata.



Dual Advantages

- **Time Efficiency:** Batch memory access accelerates graph construction.
- **Space Efficiency:** Reduces redundant storage & memory usage.

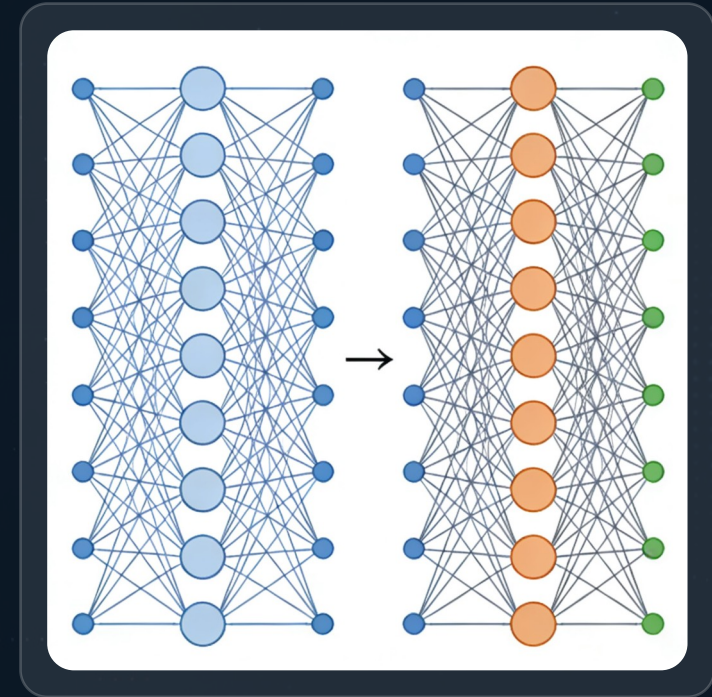


Fig: Grouped Compression of Same-degree Vertices

Experimental Setup: Testbed & Datasets



Large-scale Distributed Testbed

Deployed on 3 HPC systems with up to **79,024** compute nodes to validate scalability.



Comparison with Mainstream Systems

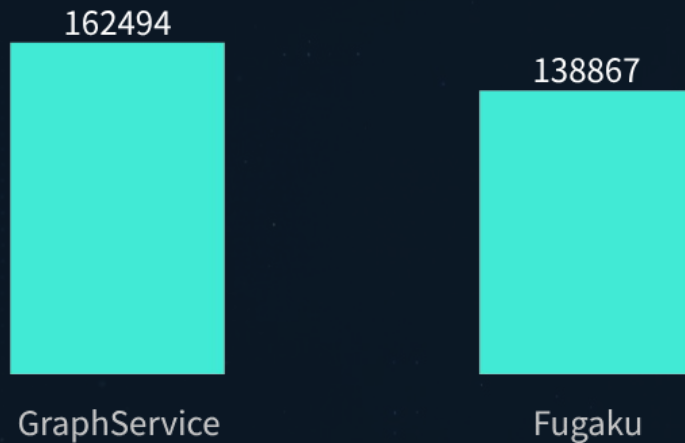
Compared with Gemini, Gluon, GraphScope, and top Graph500 supercomputing systems.

Dataset Details

Dataset	Vertices	Edges	Type
Twitter-2010	41.6M	1.47B	Real-world
Kronecker (scale=42)	4.4T	70.4T	Synthetic
Sogou	271.8B	12.3T	Real-world

Experimental Results 1: Graph500 Performance Comparison

Graph500 BFS Performance Comparison (GTEPS)



BFS Throughput

162,494 GTEPS (17% higher than champion)



SSSP Throughput

23,021 GTEPS



Graph Construction Time

Reduced by >4 orders (vs Fugaku)

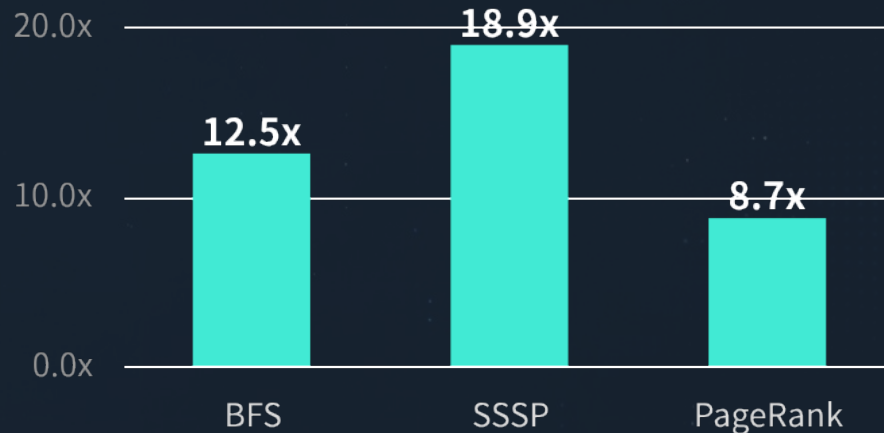
Summary: GraphService demonstrates exceptional computing performance in the authoritative Graph500 benchmark. It not only surpasses the former champion Fugaku in throughput but also achieves a disruptive breakthrough in graph construction efficiency.

Experimental Results 2: Real-world Graph Performance

Core Findings

On real-world graph datasets (Twitter, Sogou), GraphService outperforms Gemini, Gluon, GraphScope.
Across different graph algorithm workloads, the speedup of GraphService is up to **18.9x**.

Performance Speedup (vs. Best Comparison System)



Experimental Results 3: Scalability Analysis

Excellent Linear Scalability

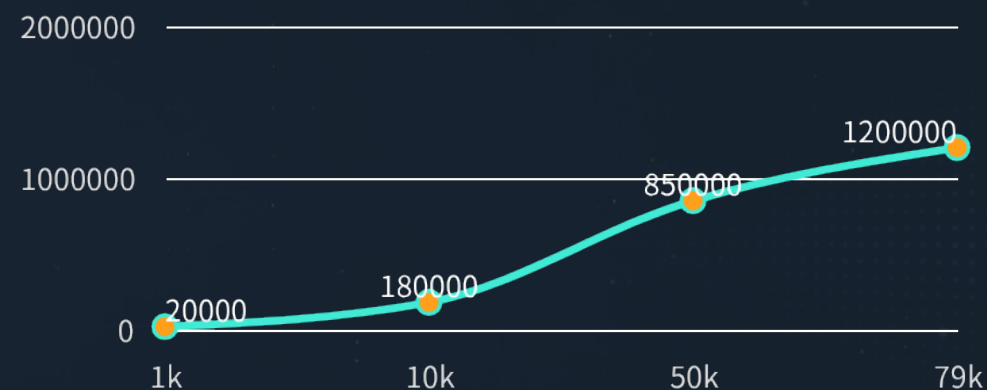
GraphService demonstrates excellent scalability. As the number of compute nodes scales from thousands to tens of thousands, the system's throughput grows nearly linearly.



Significant Improvement in Construction Efficiency

The growth rate of graph construction time is much lower than the growth rate of the number of nodes, proving its efficiency on large-scale clusters.

Throughput vs. Number of Nodes (GTEPS)



Conclusion: GraphService can continuously improve processing capacity by adding machines, fully meeting future demands for ultra-large-scale graph data.

Experimental Results 4: Compression Effect Analysis

Memory Usage Comparison



Memory Usage Significantly Reduced



Avg compression rate **>70%**, releasing system resources.

Graph Construction Time Reduced



Speed increased by **30%~50%** via batch memory access.

Conclusion: Graph compression solves the memory bottleneck for large-scale graphs and brings significant performance gains through batch access.

Related Work Comparison

The comparison shows that GraphService surpasses existing work in comprehensive performance, especially with significant advantages in graph construction efficiency and utilization of communication topology.

System	Hierarchy-aware Part.	Graph Comp.	Graph500 Perf.	Real Graph Perf.
Gemini	✗	✗	-	Baseline
Gluon	✗	✗	-	Baseline
GraphScope	✗	✗	-	Baseline
GraphService	✓	✓	1.17x	Up to 18.9x

**Core Advantages:**Unique "Hierarchy-aware Partitioning" and "Graph Compression" enable an order-of-magnitude performance breakthrough on real-world graph datasets.

Conclusion



Core Contribution: GraphService Constructor

Proposed a topology-aware graph constructor for large-scale graph applications, addressing traditional construction pain points.



Key Tech: Hierarchy-aware & Compression

Employed hierarchy-aware partitioning and efficient compression to eliminate communication bottlenecks.



Superior Performance: Order of Magnitude

Surpassed existing systems on Graph500 benchmarks and real-world data, achieving breakthrough performance.



GraphService Core Architecture Diagram

Future Work



Extend Application Scenarios & Algorithm Support

Extend GraphService to more types of graph algorithms and a wider range of application scenarios, breaking the limitations of single applications and releasing the potential of graph computing.



Optimize Process with GNN Training Frameworks

Explore integration with Graph Neural Network (GNN) training frameworks to optimize the end-to-end process from graph construction to model training, improving overall efficiency.



Optimize Compression for Dynamic Structures

Further optimize the graph compression algorithm to adapt to more complex and dynamically changing graph structures, addressing bottlenecks in large-scale graph storage and transmission.

Acknowledgements

This work is supported by the National Natural Science Foundation of China and the PDL Innovation Research Fund.

Thanks to the contributions and collaboration of all team members.



#

Q&A

Thank You

感谢聆听 · 欢迎提问